

MODÜL 01

01

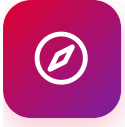
Angular & Modern Frontend Temelleri

🕒 Başlangıç · Sıfır → Junior

🔗 Bu modülde ne öğreneceğiz?

- ✓ Frontend framework'lerin hangi problemi çözmek için var olduğunu
- ✓ Reactivity (tepkiselik) kavramını ve neden modern web'in kalbi olduğunu
- ✓ Angular'ın tarihini, felsefesini ve Angular vs React vs Vue farkını
- ✓ Angular v21'in getirdiği büyük yenilikleri (zoneless, signals, Vitest, MCP)
- ✓ Kendi geliştirme ortamını kurup ilk Angular projeni çalıştırmayı

Ön koşul: HTML, CSS ve temel JavaScript bilgisi yeterli. TypeScript bilmen şart değil — Modül 2’de sıfırdan göreceğiz. Bu modülde çoğunlukla **“neden”** sorusuna odaklanıp sağlam bir zemin kuracağız.



Frontend Frameworks: Neden Var?

Bu derste tek satır Angular kodu yazmadan önce “**neden**” sorusunu cevaplayacağız. Çünkü bir aracı, hangi problemi çözmek için var olduğunu anlamadan öğrenmek, onu bir süre sonra yanlış kullanmana neden olur.

Web'in Tarihsel Hikayesi

Web başladığında (1990'lar) sayfalar tamamen **statik**ti. Sunucuda hazır duran bir HTML dosyası vardı; tıkladığın her link, sunucudan **yepyeni bir sayfanın** baştan yüklenmesi demekti. Sayfa beyazlaşıp, yeniden çizilirdi.

Sonra JavaScript geldi (1995). Sayfa yüklendikten sonra küçük işler yapılabilirdi, ama bu hâlâ “süs” seviyesindeydi. 2005 civarı **AJAX** yaygınlaştı: “sayfayı baştan yüklemekten arka planda sunucuyla konuşabilme” yeteneği. Gmail'in çıkışı dönüm noktasıydı; e-postayı okurken sayfa hiç yenilenmiyordu. Web böylece “döküman” olmaktan çıkıp “uygulama” olmaya başladı.

İşte bu noktada bir sorun doğdu: Uygulama büyüdükçe, JavaScript ile DOM'u (sayfadaki HTML elemanlarını) elle yönetmek bir kâbusa dönüştü.

Vanilla JS'in Sınırları

“Vanilla JS” = hiçbir kütüphane/framework olmadan saf JavaScript. Küçük işler için harikadır, ama uygulama büyüdükçe sorunlar çıkar. En temel örnek: ekrandaki bir sayıyı butona basınca artırmak.

```
JAVASCRIPT

let count = 0;
const display = document.getElementById('count');
const button = document.getElementById('increment');

button.addEventListener('click', () => {
  count++;
  display.textContent = count; // Veriyi değiştirince EKRANI da elle güncelleme gerek
});
```

Dikkat et: `count` değişkenini değiştirmek **yetmiyor**; ayrıca `display.textContent` 'i de elle güncelliyorsun. Yani “veri” ile “ekran” arasındaki senkronizasyonu **sen** yönetiyorsun. Bu sayı 3 yerde gösteriliyorsa, 3 yeri de elle güncellersin. Unuttuğun an ekranda yanlış veri kalır.

Buna ek olarak kod tekrarı (50 kartı elle oluşturmak) ve durum (state) takibi de büyüdükçe sürdürülemez hale gelir.

Tek cümleyle

Vanilla JS'te “veri değişti, ekranı güncelle” işini hep **sen elle** yaparsın. Framework'ler işte tam bu işi senin yerine yapar.

SPA (Single Page Application)

SPA = Tek Sayfa Uygulaması. Modern web uygulamalarının çoğu böyledir (Gmail, Trello, Spotify web...). Tarayıcı **bir kez** boş bir HTML kabuğu ve büyük bir JS paketi yükler; sonrası JavaScript tarafında halledilir. Sunucudan sadece **veri** (genelde JSON) çekilir, koca HTML sayfaları değil.

	Geleneksel (MPA)	SPA
Sayfa geçişi	Sunucudan yeni sayfa	JavaScript ekranı değiştirir
İlk yükleme	Hızlı	Daha yavaş (büyük JS paketi)
Sonraki geçişler	Yavaş (her seferinde yeni yük)	Çok hızlı (anlık)
Hissiyat	Web sitesi	Masaüstü/mobil uygulama gibi

Component-Based Architecture

Modern framework'lerin en önemli fikri: **arayüzü, kendi içinde bütün, küçük, yeniden kullanılabilir parçalara (component) böl.** Bir e-ticaret sayfasını şöyle ayırabilirsin:

```
SayfaComponent
├── HeaderComponent
│   ├── LogoComponent
│   ├── AramaCubuguComponent
│   └── SepetIkonuComponent
├── UrunListesiComponent
│   ├── UrunKartiComponent ← bu 50 kez tekrar kullanılır
│   ├── UrunKartiComponent
│   └── ...
└── FooterComponent
```

Her component kendi **HTML**'ini, **CSS**'ini ve **mantığını** (TypeScript) içinde taşır. **UrunKartiComponent** 'i bir kez yazar, 50 kez kullanırsın; bir yeri değiştirtince hepsi değişir. Bu yaklaşım hem kod tekrarını önler hem de büyük problemi küçük, anlaşılır parçalara böler.

Reactivity Nedir?

Bu, tüm modern framework'lerin kalbidir ve Angular v21'in en çok yatırım yaptığı konudur. **Reactivity (tepkiselik) = “Veri değiştiğinde, o veriye bağlı ekranın otomatik güncellenmesi.”**

```
// Angular v21 (Signals ile) – sözde kod, detayı Modül 5'te
count = signal(0);

artir() {
  this.count.set(this.count() + 1);
  // BİTTİ. Ekranı sen güncellemiyorsun.
  // count'a bağlı her yer otomatik güncellenir.
}
```

Excel metaforu

Excel'de B1 hücresine `=A1*2` yazarsan, A1'i değiştirdiğinde B1 otomatik güncellenir. A1 “değiştirdiği için” B1 “tepki verir.” Reactivity tam olarak budur. Angular'ın **Signal** sistemi de aynen böyle çalışır.

Vanilla JS vs Framework

Konu	Vanilla JS	Framework
Veri-ekran senkronu	Elle (sen yaparsın)	Otomatik (reactivity)
Kod organizasyonu	Sana kalmış (kaos riski)	Component yapısı
Yeniden kullanım	Zor, kopyala-yapıştır	Component ile kolay
Routing / Form	Elle yazılır	Hazır araçlar
Öğrenme eğrisi	Başta düşük	Başta yüksek
Büyük projede	Sürdürülemez	Sürdürülebilir

Önemli

Framework “her zaman daha iyi” değildir. 3 sayfalık tanıtım sitesi için Angular kullanmak, çiviye balyozla çakmaktır. Ama gerçek bir uygulama yazıyorsan, framework olmadan bir süre sonra duvara toslarsın.

Bu Eğitimde Kullanılacak Felsefe

1. **Önce “neden”, sonra “nasıl”.** Her özelliği hangi problemi çözdüğünü anlayarak öğreneceğiz. Ezber yok.
2. **Modern Angular.** Eski yaklaşımları sadece tanıman için göstereceğiz; yazarken hep v21'in modern yolunu kullanacağız.
3. **Bol örnek + mini proje.** Kod ancak yazılarak öğrenilir.
4. **Anti-pattern'ları görmek.** “Şunu yapma”, “şunu yap” kadar değerlidir.

🔍 Mini Quiz

1. Vanilla JS ile framework arasındaki en temel fark (veri değişince ekranın güncellenmesi açısından) nedir?
2. SPA'da sunucudan genellikle ne çekilir: koca HTML sayfaları mı, JSON verisi mi?
3. Component-based architecture'ın iki temel faydası nedir?
4. Reactivity'yi bir Excel hücresi metaforuyla açıkla.

🔑 Cevaplar

1. Vanilla JS'te senkronizasyonu geliştirici elle yapar; framework bunu reactivity ile otomatik yapar.
2. Genellikle JSON verisi. HTML kabuğu bir kez yüklenir, sonrası JavaScript'le yönetilir.
3. (a) Yeniden kullanılabilirlik, (b) Büyük problemi küçük, yönetilebilir parçalara bölme.
4. $B1 = A1 * 2$ ise A1 değişince B1 otomatik güncellenir; veri değişince ona bağlı olan tepki verir.

❌ Anti-Patterns

- ❌ **Her şey için framework kullanmak.** Statik bir landing page için Angular kurmak gereksiz karmaşıklıktır.
- ❌ **Framework'ü “sihir” sanmak.** Reactivity'nin arkasında bağımlılık takibi yapan bir sistem var; anlamadan kullanırsan performans sorunlarını çözemezsin.
- ❌ **Vanilla JS alışkanlıklarını taşımak.** Angular içinde `document.getElementById` ile DOM'u elle değiştirmek framework'ün avantajını çöpe atar.

✅ Bu Dersten Çıkarılacaklar

- ✓ Web “döküman”dan “uygulama”ya evrildi; bu evrim framework'lere ihtiyaç doğurdu.
- ✓ Vanilla JS'in temel sınırı: veri-ekran senkronizasyonunu elle yapmak.
- ✓ Framework'lerin üç büyük fikri: component yapısı, reactivity, hazır araçlar.
- ✓ Reactivity = veri değişince ona bağlı ekranın otomatik güncellenmesi. Angular bunu Signals ile yapar.
- ✓ Framework bir araçtır; her işe değil, doğru işe kullanılır.



AngularJS — Her Şeyin Başlangıcı

2010'da Google, **AngularJS** (1.x) adında devrimsel bir framework yayınladı. “İki yönlü veri bağlama” fikrini popülerleştirdi: HTML'e `ng-model`, `ng-repeat` gibi özel öznitelikler ekliyordun ve sihir gibi çalışıyordu. Ama mimari sorunları vardı; “digest cycle” mekanizması uygulama büyüdükçe yavaşlıyordu ve yapısı modern JavaScript ile mobil çağ için tasarlanmamıştı.

⚠ Karışıklık uyarısı

“**AngularJS**” (1.x) ile bugünkü “**Angular**” (2+) farklı framework'lerdir. İsim benzer ama içleri tamamen ayrıdır; birinden diğerine “güncelleme” yapamazsın. Bugün “Angular” dendiğinde kimse AngularJS'i kastetmez — o teknoloji 2021'de resmen emekli oldu.

Büyük Yıkım ve Yeniden Doğuş — Angular 2

2016'da Google, AngularJS'i yamalamak yerine **sıfırdan** yeni bir framework yazdı: **Angular 2** (sonradan sadece “Angular”). Eski projeler doğrudan taşınmadığı için tartışma yarattı ama karar doğrudu: yeni Angular baştan TypeScript ile, component mimarisiyle, modern araçlarla tasarlandı. Bugün kullandığımız Angular, o yeniden doğuşun evrilmiş hâlidir.

Angular'ın Felsefesi — “Opinionated”

Angular'ı React ve Vue'dan ayıran en temel özellik: Angular “**opinionated**” (**görüşlü**) bir framework'tür. Sana “işleri şöyle yap” der; klasör yapısı, routing, form, HTTP, test — hepsinin resmî bir yolu vardır ve kutudan çıkar (“batteries included”). React ise sadece bir arayüz kütüphanesidir; routing, state vb. seçimleri sana bırakır.

	Avantaj	Dezavantaj
Angular (opinionated)	Büyük takımlarda tutarlılık, her araç kutudan çıkar	Dik öğrenme eğrisi, daha az esneklik
React (kütüphane)	Çok esnek, hafif başlangıç	Her takım kendi “Angular'ını” kurar; tutarsızlık riski

Angular'ın bu felsefesi, özellikle **kurumsal ve büyük ekipli** projelerde neden tercih edildiğini açıklar.

Angular'ın Versiyon Tarihi

Angular **her yıl iki büyük sürüm** çıkarır (yaklaşık 6 ayda bir). Önemli dönüm noktaları:

Sürüm	Tarih	Öne çıkan
AngularJS (1.x)	2010	Başlangıç (artık emekli)
Angular 2	2016	Sıfırdan yeniden yazım, TypeScript
Angular 14	2022	Standalone (öncül), typed forms
Angular 16	2023	Signals (ilk hâli)
Angular 17	2023	Yeni control flow, @defer
Angular 19	2024	linkedSignal, resource API, zoneless (öncül)
Angular 20	2025 (Mayıs)	Birçok özelliğin stabilleşmesi
Angular 21	2025 (20 Kasım)	Zoneless varsayılan, Vitest, Signal Forms, Aria, MCP
Angular 22	~2026 (Mayıs)	(yakında — bu eğitim v21 üzerine)

Angular her major sürüme **18 ay** destek verir ve **ng update** ile otomatik migration sunar — yani güncel kalmak çok kolaydır.

❗ Not

Bu eğitim hazırlanırken Angular 22'nin çıkışı çok yakındı. Ama müfredatın v21 üzerine kurulu ve v21'in getirdikleri (zoneless, signals, signal forms) bir süre daha sektörün ana gündemi olacak. v22'ye geçmek istediğinde **ng update** ile sorunsuz ilerlersin.

Angular Kim Tarafından Kullanılıyor?

Arkasında **Google** var ve Google kendi devasa ürünlerinde Angular kullanıyor (Google Cloud Console, Firebase Console, Google Ads...). Ayrıca Microsoft'un bazı bölümleri, Deutsche Bank, Forbes, Delta, IBM gibi birçok kurumsal şirket. Örüntü net: Angular **büyük, uzun ömürlü, kurumsal** uygulamalarda güçlüdür.

Angular'ın Geleceği & Neden v21?

Angular'ın yönü çok net: (1) **Signals'a tam geçiş**, (2) **zoneless dünya**, (3) **AI çağına entegrasyon** (MCP), (4) daha iyi geliştirici deneyimi. v21 ise bir **dönüm noktası** sürümüdür: yıllardır “deneysel” olan birçok büyük fikir burada standart/varsayılan oldu (zoneless varsayılan, Vitest varsayılan, olgunlaşmış signals). Yani v21'i öğrenmek, “geleceğin Angular'ını bugünden” öğrenmektir; kafanda eski alışkanlıklar oluşmaz.

🔍 Mini Quiz

1. “AngularJS” ile “Angular” aynı şey midir? Neden?
2. Angular'ın “opinionated” olması ne demek? Bir avantaj ve bir dezavantaj söyle.
3. Angular yılda kaç büyük sürüm çıkarır ve sürüm geçişini kolaylaştıran komut nedir?
4. v21'i bu eğitim için “dönüm noktası” yapan en az iki yenilik nedir?

🔑 Cevaplar

1. Hayır. AngularJS (1.x) ile Angular (2+) tamamen farklı framework'lerdir; biri diğerine güncellenemez.
2. İşleri yapmanın resmî bir yolunu dayatır. Avantaj: büyük takımlarda tutarlılık. Dezavantaj: dik öğrenme eğrisi.
3. Yılda iki büyük sürüm (~6 ayda bir). Geçiş `ng update` kolaylaştırır.
4. Herhangi ikisi: zoneless varsayılan, Vitest varsayılan, Signal Forms, Angular Aria, MCP, olgunlaşmış signals.

⊗ Anti-Patterns

- ⊗ **AngularJS kaynaklarıyla Angular öğrenmek.** Eski `ng-model`, `$scope` içerikleri bugünkü Angular için geçersizdir.
- ⊗ **Eski (NgModule dönemi) örnekleri kopyalamak.** 2022 öncesi örnekler `NgModule` kullanır; v21'de standalone kullanacağız.
- ⊗ **Sürüm numarasından korkmak.** Migration araçları sayesinde güncel kalmak kolaydır; temel kavramlar sürümler arası sabittir.

✅ Bu Dersten Çıkarılacaklar

- ✓ AngularJS (2010) ve Angular (2016+) farklı framework'lerdir; bugün “Angular” = 2+ sürümü.
- ✓ Angular opinionated'tır: tutarlılık sağlar ama öğrenme eğrisi diktir — kurumsal projelerde güçlü.
- ✓ Yılda 2 büyük sürüm, her birine 18 ay destek, `ng update` ile kolay geçiş.
- ✓ Angular'ın yönü: signals + zoneless + AI entegrasyonu.
- ✓ v21, olgunlaşan fikirlerin standartlaştığı bir dönüm noktası sürümüdür.



Karşılaştırma Senaryosu

Üç framework'ü konuşmak yerine **aynı işi** üçünde de yazıp görelim: bir sayaç. Ekranda bir sayı ve bir buton var; basınca sayı 1 artıyor. Bu küçük örnek her framework'ün “veriyi nasıl tutuyor, ekranı nasıl güncelliyor, olayları nasıl dinliyor” felsefesini gösterir.

Not

React ve Vue kodunu anlamak zorunda değilsin. Amaç onları öğretmek değil; Angular'ın yaklaşımını kıyaslayarak daha net görmek.

React Versiyonu

JSX

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Sayı: {count}</p>
      <button onClick={() => setCount(count + 1)}>Artır</button>
    </div>
  );
}
```

React'te HTML, JavaScript'in içinde yazılır (JSX). State `useState` ile tutulur, `setCount` ile değiştirilir.

Vue Versiyonu

VUE

```
<script setup>
import { ref } from 'vue';
const count = ref(0);
</script>

<template>
  <div>
    <p>Sayı: {{ count }}</p>
    <button @click="count++">Artır</button>
  </div>
</template>
```

Vue, `<script>` ve `<template>` bölümlerini ayırır. State `ref` ile tutulur. Söz dizimi oldukça sadedir.

Angular Versiyonu (v21)

TYPESCRIPT

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-counter',
  template: `
    <div>
      <p>Sayı: {{ count() }}</p>
      <button (click)="artir()">Artır</button>
    </div>
  `,
})
export class Counter {
  count = signal(0);

  artir() {
    this.count.set(this.count() + 1);
  }
}
```

Angular'da bir component, `@Component` ile işaretlenmiş bir **class**'tır. State `signal()` ile tutulur, template'te `{{ count() }}` ile okunur (parantezlere dikkat — signal bir fonksiyon gibi çağrılır). Olaylar `(click)` ile dinlenir. Her parçayı Modül 4 ve 5'te öğreneceğiz.

Yan Yana Karşılaştırma

Konu	React	Vue	Angular v21
Dil	JS/TS + JSX	JS/TS + template	TypeScript zorunlu
Component	Fonksiyon	.vue dosyası	@Component class
State	useState	ref	signal()
Olay	onClick	@click	(click)
Yapı	Esnek (kütüphane)	Orta	Opinionated

“Bu Kod Bana Ne Söylüyor?”

- Üçü de aynı temel fikri paylaşıyor:** veriyi tut, ekranda göster, değiştir, ekran otomatik güncellensin (reactivity). Birini öğrenince diğerlerine geçiş kolaydır.
- Angular en “yapılı” olanıdır.** Class, decorator, tip zorunluluğu... Başta daha fazla “tören” gibi görünür ama büyük projede kaybolmanı engeller.

3. **Angular TypeScript'i zorunlu kılar.** Bu yüzden Modül 2'yi tamamen TypeScript'e ayırdık.

“Hangisini Öğrenmeliyim?”

Dürüst cevap: **üçü de iyi seçimdir** ve hiçbirini ölmüyor. Tercih yaparken:

- **Angular** → büyük/kurumsal, uzun ömürlü projeler; güçlü yapı, net kurallar.
- **React** → en büyük ekosistem ve iş havuzu, maksimum esneklik.
- **Vue** → en yumuşak öğrenme eğrisi, sadelik, hızlı başlangıç.

Sen Angular'ı seçtin — özellikle güçlü mimari ve kurumsal kariyer için iyi bir tercih. Üstelik Angular öğrenince diğerlerinin mantığını da kavramış olacaksın.

🔍 Mini Quiz

1. Üç framework'ün de paylaştığı ortak temel fikir nedir?
2. Angular'da state tutmak için ne kullanılır, React'te ne kullanılır?
3. Hangi framework TypeScript'i zorunlu kılar?
4. “Opinionated tam framework” tanımını en çok hangisine uyar?

🔑 Cevaplar

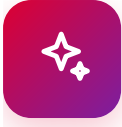
1. Reactivity: veriyi tut, ekranda göster, değiştir, ekran otomatik güncellensin.
2. Angular: `signal()`. React: `useState`.
3. Angular.
4. Angular.

⊗ Anti-Patterns

- ⊗ “En iyi framework hangisi?” tartışmasına saplanmak. Doğru soru “bu proje/takım için hangisi uygun”.
- ⊗ Angular'ı React gibi kullanmaya çalışmak. Her framework'ün kendi felsefesi var; zorlamak zarar verir.
- ⊗ Signal'i parantezsiz okumak. `{{ count }}` değil, `{{ count() }}`. Signal bir fonksiyondur; değerini almak için çağrılır.

✅ Bu Dersten Çıkarılacaklar

- ✓ Aynı sayaç üç framework'te de benzer mantıkla yazılır: tut → göster → değiştir → otomatik güncelle.
- ✓ Angular en yapılı (class + decorator + zorunlu TypeScript) olandır; bu yapı büyük projelerde güç verir.
- ✓ Angular'da: state = `signal()`, olay = `(click)`, gösterim = `{{ deger() }}`.
- ✓ Framework seçimi proje ve takıma bağlıdır; Angular kurumsal ve büyük projelerde parlar.



Angular v21'in “Big Picture” Yenilikleri

Bu ders, eğitimin geri kalanının **haritasıdır**. Her özelliği burada derinlemesine öğrenmeyeceğiz (onlar kendi modüllerinde gelecek); sadece “v21'de ne değişti ve neden önemli” resmini çizeceğiz.

❗ v21 bir “geçiş” versiyonu değil

Yeni bir Angular projesi açtığında artık varsayılan olarak **zoneless** çalışır, testlerin **Vitest** ile gelir, reactivity signal-tabanlıdır. Geleceğin Angular'ı artık bugünün varsayılanı.

Zoneless Change Detection

v21'in en büyük değişikliği. “**Change detection**” (**değişiklik tespiti**), Angular'ın “verim değişti mi, ekranı güncellemeli miyim?” sorusunu cevaplama mekanizmasıdır.

Eski yöntem (zone.js): Angular yıllarca `zone.js` kullandı. Bu kütüphane tarayıcıdaki **her olayı** (tık, timer, HTTP) gizlice izler ve “bir şey oldu, her şeyi kontrol edelim” derdi. Çalışıyordu ama ekstra yük getiriyor, gereğinden fazla kontrol yapıp performansı düşürebiliyordu.

Yeni yöntem (zoneless): v21'de yeni projeler `zone.js` olmadan gelir. Angular değişikliği **Signals sayesinde** anlar — bir signal değişince zaten tam olarak neyin güncelleneceğini bilir, her şeyi taramaz.

🗨️ Senin için sonucu

Daha hızlı uygulamalar (özellikle INP gibi metriklerde), daha küçük paket boyutu, daha öngörülebilir davranış ve `async/await` 'i rahat kullanım. Detay → **Modül 22**.

Signal Forms

Form yönetimi, web uygulamalarının kalbidir. Angular'ın iki eski form yaklaşımı (template-driven ve reactive) hâlâ var. v21 üçüncü, modern bir seçenek ekledi: **Signal Forms** — formları da signal-tabanlı reactivity üzerine kurar; reactive forms'un gücünü template-driven'ın sadeliğiyle birleştirmeyi hedefler.

⚠️ Önemli

Signal Forms şu an **deneysel (experimental)**. Yani API'si değişebilir; nitekim v21.1'de bir directive'in adı `[field]` 'dan `[formField]` 'a değişti. Üretim projesinde kullanmadan önce iki kez düşün; öğrenmek için harika. Detay → **Modül 14**.

Angular Aria

a11y (accessibility = erişilebilirlik), arayüzünün ekran okuyucu kullananlar veya klavyeyle gezenler tarafından da kullanılabilmesidir; hem etik hem çoğu ülkede yasal zorunluluktur. **Angular Aria**, erişilebilirliği baştan doğru yapmış

“headless” (stilsiz) component'ler sunar: davranış ve erişilebilirlik hazır gelir, görünümü (CSS'i) sen giydirirsin. Şu an **Developer Preview**. Detay → **Modül 19**.

Vitest — Karma'nın Sonu

Angular yıllarca **Karma** + **Jasmine** ile test çalıştırdı; Karma yaşlandı ve yavaştı. v21'de **Vitest** varsayılan test çalıştırıcısı oldu: modern, çok daha hızlı, zoneless dünyayla uyumlu. (Jasmine söz dizimi hâlâ destekleniyor; Karma emekli edildi, Jest ve Web Test Runner ise deprecated olup v22'de kaldırılacak.) Detay → **Modül 21**.

MCP Server — AI Çağına Entegrasyon

Problem: AI asistanlar (Claude, Copilot, Gemini...) genelde eski Angular bilgisiyle eğitilmiştir; sen v21 isterken sana NgModule'lü eski kod önerebilir. **Çözüm — MCP (Model Context Protocol) Server:** AI asistanlarının **güncel** Angular bilgisine ve **senin projenin yapısına** erişebileceği bir köprü. Böylece AI, eski/genel kod yerine projene ve v21 standartlarına uygun kod üretir. Angular, MCP'yi resmî benimseyen ilk büyük framework; v21'de 7 araçla geliyor. Detay → **Modül 3, Ders 3.6**.

Tailwind CSS & Diğer Yenilikler

v21, **Tailwind** kurulumunu kolaylaştıran resmî bir *schematic* ekledi: yeni proje açarken ya da hazır projeye `ng add` ile tek komutla, doğru yapılandırmayla (`tailwindcss` , `@tailwindcss/postcss` , `postcss`) ekleyebiliyorsun. Diğer yenilikler:

- **Daha iyi hata izleri** (error stack traces) → kolay hata ayıklama.
- **Component-level hydration** kontrolü (SSR — Modül 25).
- **Gelişmiş template tip çıkarımı** → hatalar daha erken yakalanır.
- **HammerJS desteği kaldırıldı** → modern Pointer Events API kullanılıyor.
- Genel build/araç hızlanmaları ve `ng serve` iyileştirmeleri.

❗ Bu yenilikler senin için ne demek?

Daha hızlı uygulamalar (zoneless + signals), daha az boilerplate, daha hızlı test (Vitest), daha akıllı AI yardımı (MCP) ve **modern bir başlangıç** — eski alışkanlıklar öğrenmeden doğrudan geleceğin Angular'ıyla başlıyorsun.

🔍 Mini Quiz

1. “Change detection” tek cümleyle nedir?
2. Zoneless dünyada Angular değişikliği neyin sayesinde anlar?
3. Signal Forms şu an stabil mi, deneysel mi? Pratikte ne anlama gelir?
4. MCP Server hangi problemi çözer?
5. Karma'nın yerini hangi test aracı aldı?

🔑 Cevaplar

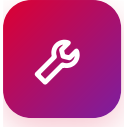
1. Angular'ın “verim değişti mi, ekranı güncellemeli miyim?” sorusunu cevaplama mekanizması.
2. Signals sayesinde — bir signal değişince Angular tam olarak neyin güncelleneceğini bilir.
3. Deneysel. API'si değişebilir; üretimde dikkatli, öğrenmek için uygun.
4. AI'nın eski/genel kod yerine güncel v21 ve projenin yapısına uygun kod üretmesini sağlar.
5. Vitest.

⊗ Anti-Patterns

- ⊗ **Deneysel özellikleri üretimde gözü kapalı kullanmak.** Signal Forms gibi “experimental” şeyler değişebilir.
- ⊗ **Zoneless'i “her şeyi otomatik halleder” sanmak.** Zoneless signals'a dayanır; reactivity'yi doğru kurmazsan ekran güncellenmeyebilir.
- ⊗ **a11y'yi “sonra hallederim” diye ertelemek.** Erişilebilirlik baştan tasarlanır.

✅ Bu Dersten Çıkarılacaklar

- ✓ v21, büyük fikirlerin varsayılan hâle geldiği bir dönüm noktası sürümü.
- ✓ Zoneless (signals'a dayalı, daha hızlı change detection) yeni projelerde varsayılan.
- ✓ Signal Forms (deneysel) ve Angular Aria (preview) yeni nesil araçlar.
- ✓ Vitest test için varsayılan (Karma emekli).
- ✓ MCP Server, AI'nın güncel ve projeye uygun kod üretmesini sağlar; Tailwind tek komutla kurulur.



Teoriyi bitirdik. Bu derste bilgisayarını Angular'a hazır hale getirip **ilk projeni çalıştıracaksın**. Adımları sırayla takip et.

Geliştirme Ortamımızda Neler Olacak?

1. **Node.js** — JavaScript'i tarayıcı dışında çalıştıran motor; tüm araçlar bunun üzerinde döner.
2. **Angular CLI** — Angular'ın komut satırı aracı (**ng**): proje oluşturma, çalıştırma, kod üretme.
3. **VS Code** — kod editörümüz (ücretsiz, en yaygın).
4. **Git** — kod versiyonlama. Profesyonel iş için olmazsa olmaz.

Ayrıca **Angular DevTools** adlı bir tarayıcı eklentisi de kuracağız.

Node.js Kurulumu

Angular v21 şu Node.js sürümlerini destekler: **20.19+**, **22.12+** veya **24.x**. En güvenli seçim çift numaralı bir **LTS** sürümüdür.

Öneri

Node **22 LTS** kur. Hem v21 ile tam uyumlu hem uzun süre desteklenecek. (Angular tek numaralı Node sürümlerini — 21, 23 — resmî desteklemez.)

Kurulum: **nodejs.org** adresinden **LTS** sürümünü indir ve kur. Çoklu proje için **nvm** (Node Version Manager) idealdir ama başlangıçta şart değil. Kontrol et:

BASH

```
node --version    # örn. v22.12.0 görmelisin
npm --version     # npm de Node ile birlikte gelir
```

npm (Node Package Manager), paket/kütüphane yöneticisidir; Angular dahil tüm bağımlılıkları bununla kurarsın.

Angular CLI Kurulumu

CLI'ı global (tüm sistemde kullanılabilir) kuruyoruz:

BASH

```
npm install -g @angular/cli

ng version    # CLI sürümünü ve ortamı gösterir – 21.x olmalı
```

Sürüm 21.x değilse: **npm install -g @angular/cli@latest**.

İpucu

`ng` komutu “not found” derse, Node'un global paket klasörü PATH'te olmayabilir. Terminali kapatıp açmak çoğu zaman çözer.

VS Code Kurulumu ve Uzantılar

code.visualstudio.com adresinden kur. Sonra şu uzantıları yükle:

- **Angular Language Service** (resmî) — template'lerde otomatik tamamlama, hata yakalama, “tanıma git”. **Mutlaka** kur.
- **Prettier** — kodu otomatik biçimlendirir.
- **ESLint** — kod kalitesi/hata denetimi.
- **Material Icon Theme** (opsiyonel) — dosya türlerine güzel ikonlar.

VS Code Ayarları

Kaydedince otomatik biçimlendirme çok zaman kazandırır (User Settings JSON):

JSON

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },
  "files.autoSave": "onFocusChange"
}
```

Git Kurulumu

git-scm.com adresinden kur, kimliğini bir kez tanımla:

BASH

```
git config --global user.name "Adın Soyadın"
git config --global user.email "email@ornek.com"
```

İyi haber: `ng new` ile oluşturulan her proje otomatik bir git deposu olarak başlar; ekstra bir şey yapmana gerek kalmadan geçmişin tutulur.

Angular DevTools Kurulumu

Chrome/Edge için bir tarayıcı eklentisidir. Web Store'da “**Angular DevTools**” ara ve ekle. Çalışan uygulamanın component ağacını görmeni, veri durumlarını incelemeni ve performansı ölçmeni sağlar. Detaylı kullanım → **Modül 3, Ders 3.5.**

İlk Angular Projemizi Oluşturma

Projelerini koymak istediğin klasöre git ve:

```
ng new merhaba-angular
cd merhaba-angular
ng serve
```

BASH

`ng new` birkaç soru sorar (stil formatı, SSR vb.), dosyaları oluşturur ve bağımlılıkları kurar (birkaç dakika). `ng serve` geliştirme sunucusunu başlatır; tarayıcıda **http://localhost:4200** adresini aç — ilk Angular uygulaman karşında!

❗ CLI'nin sorduğu sorular

Stil formatı (CSS/SCSS — **SCSS** önerilir), SSR/SSG (başlangıçta **No** diyebilirsin), AI araçları yapılandırması vb. v21'de yeni projeler zaten **zoneless** ve **Vitest** ile gelir. Soruları Modül 3, Ders 3.1'de tek tek inceleyeceğiz.

Hızlı Bir Düzenleme — Hot Reload Görelim

`ng serve` çalışırken (kapatma!) VS Code'da projeyi aç (`code .`), `src/app/app.html` dosyasına bir başlık ekle:

```
<h1>Merhaba, Angular'a hoş geldin!</h1>
```

HTML

Kaydet (`Ctrl+S`) ve tarayıcıya bak. **Sayfayı yenilemeden** değişiklik anında yansır. Buna **Hot Reload** denir — kaydet, anında gör.

Geliştirme Workflow'u & Angular DevTools

Tarayıcıda `F12` → **Angular** sekmesi → **Components** bölümünde component ağacını görürsün. Günlük akışın:

1. `ng serve` ile sunucuyu başlat (gün boyu açık bırak).
2. Kodu yaz/kaydet → hot reload ile anında gör.
3. Yeni component/service için `ng generate ...` (Modül 3).
4. Git ile düzenli commit at.
5. Tarayıcı + Angular DevTools ile hata ayıkla.

Sık Karşılaşılan Problemler

Problem	Çözüm
<code>ng: command not found</code>	CLI kurulmamış/PATH sorunu. <code>npm install -g @angular/cli</code> + terminali yeniden başlat.
Node sürüm uyarısı	v21 desteklemediği Node'da durur. Node'u 22 LTS'e güncelle.
<code>Port 4200 is already in use</code>	<code>ng serve --port 4300</code> ile başka porttan başlat.
<code>npm install</code> yavaş/hata	<code>npm cache clean --force</code> sonra tekrar kur.
Template'te otomatik tamamlama yok	Angular Language Service uzantısını kur.

Pratik Komutlar Sözlüğü

	BASH
<code>node --version</code>	<i># Node sürümü</i>
<code>ng version</code>	<i># Angular CLI sürümü ve ortam</i>
<code>ng new <proje-adi></code>	<i># Yeni proje oluştur</i>
<code>ng serve</code>	<i># Geliştirme sunucusu (localhost:4200)</i>
<code>ng serve --port 4300</code>	<i># Farklı porttan başlat</i>
<code>ng generate component <ad></code>	<i># Yeni component (kısa: ng g c <ad>)</i>
<code>ng build</code>	<i># Production için derle</i>
<code>ng update</code>	<i># Angular'ı güncelle</i>
<code>code .</code>	<i># Bulunduğun klasörü VS Code'da aç</i>

Mini Quiz

- Angular v21 için önerilen Node.js sürümü hangisidir ve neden çift numaralı LTS seçeriz?
- Angular CLI'ı global kurmak için hangi komut kullanılır?
- `ng serve` ne yapar ve uygulamayı hangi adreste açar?
- “Hot Reload” nedir?
- VS Code'da mutlaka kurulması gereken Angular uzantısı hangisidir?

Cevaplar

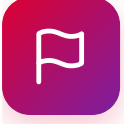
- Node 22 LTS. Çift numaralı sürümler LTS (uzun destek) alır; Angular tek numaralı sürümleri resmî desteklemez.
- `npm install -g @angular/cli`
- Geliştirme sunucusunu başlatır; uygulamayı `http://localhost:4200` adresinde açar.
- Kodu kaydedince, sayfayı elle yenilemeden değişikliğin tarayıcıda anında görünmesi.
- Angular Language Service.

⊗ Anti-Patterns

- ✗ **Desteklenmeyen Node sürümü kullanmak.** v21 desteklemediği Node'da çalışmaz; baştan doğru sürümü kur.
- ✗ **CLI'ı global kurmamak / sürümünü kontrol etmemek.** Eski CLI eski yapıda proje üretir; `ng version` ile 21.x doğrula.
- ✗ `ng serve` 'ü her değişiklikte kapatıp açmak. Gerek yok; açık bırak, hot reload halleder.
- ✗ **DOM'u `console.log` ile didiklemek.** Angular DevTools tam bu iş için var.

☑ Bu Dersten Çıkarılacaklar

- ✓ Ortam = Node.js (22 LTS) + Angular CLI (21.x) + VS Code (+ Angular Language Service) + Git.
- ✓ Proje oluşturma: `ng new` , çalıştırma: `ng serve` → `localhost:4200` .
- ✓ Hot Reload sayesinde kaydet-gör döngüsü anlaktır.
- ✓ Angular DevTools, uygulamayı incelemek için tarayıcı eklentidir.
- ✓ v21'de yeni projeler zaten zoneless + Vitest ile gelir — modern başlangıç hazır.



Tebrikler — ilk modülü tamamladın! Tek satır karmaşık kod yazmadan, Angular'ı **anlamak** için sağlam bir zemin kurduk.

Kavramsal olarak öğrendiklerin

- Web neden “döküman”dan “uygulama”ya evrildi ve bu neden framework'lere ihtiyaç doğurdu.
- Vanilla JS'in temel sınırı: veri-ekran senkronizasyonunu elle yapmak. Çözüm: **reactivity**.
- Üç büyük fikir: **component mimarisi**, **reactivity**, hazır araçlar.
- Angular'ın kimliği: **opinionated** bir tam framework — büyük/kurumsal projelerde güçlü.
- AngularJS (1.x) ≠ Angular (2+); bugün “Angular” modern framework'tür.
- Angular vs React vs Vue: hepsi aynı reactivity fikrini paylaşır; Angular en yapılı ve TypeScript-merkezli olandır.

v21'in büyük resmi

- **Zoneless** change detection varsayılan (signals'a dayalı, daha hızlı).
- **Signal Forms** (deneysel), **Angular Aria** (preview), **Vitest** (test varsayılanı), **MCP Server** (AI), kolay **Tailwind**.

Pratik olarak yaptıkların

- Node.js + Angular CLI + VS Code + Git kurdun.
- `ng new` ile ilk projeni oluşturup `ng serve` ile çalıştırdın.
- Hot Reload'u ve Angular DevTools'u ilk kez gördün.

❓ Sırada ne var?

Modül 2'de TypeScript'e gireceğiz. Angular'ın dili TypeScript ve onu iyi bilmek, Angular'ı rahat yazmanın ön koşulu. Tipler, interface'ler, generics, utility types ve type guards'ı sıfırdan, Angular bağlamında öğreneceğiz.

🗉 Kendini test et

Bu modülü bir arkadaşına 5 dakikada anlatabilir misin? “Framework neden var, reactivity nedir, Angular neden opinionated, v21'de ne yeni?” — bu dört soruyu kendi cümlelerinle cevaplayabiliyorsan, modülü gerçekten sindirmişsin demektir.

Takıldığın her yeri sor — sonra Modül 2'ye geçeriz. Kolay gelsin!